B.TECH INFORMATION TECHNOLOGY · MINI PROJECT · AY 2026-27

Intelligent Formal Document to Analytics Dashboard: System Modules

The project code split into nine functional modules. Each module has a title, a description and its code.

Sources: app.py, ingestion/, extraction/, schema/, analytics/, visualization/, ui/ (github.com/pranavwaghmare/analyticsdashboard)

TEAM MEMBERS

1. Pranav Waghmare 2. Vedant Ghodekar
3. Harsh Prajapati 4. Souvik Routh

TEAM GUIDE

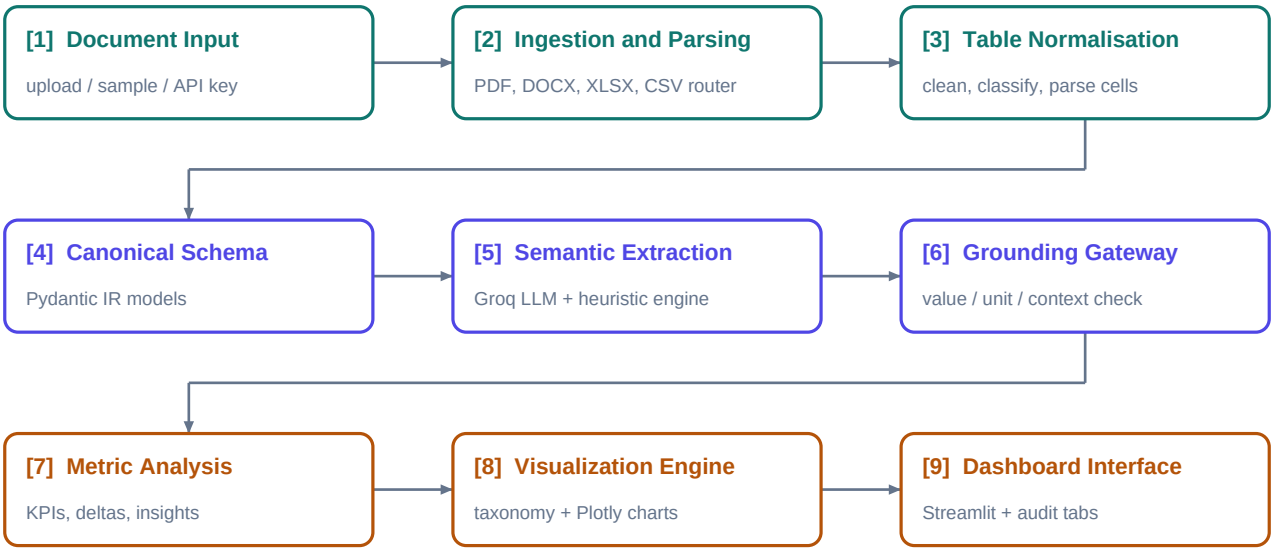
Prof. Neha Kudu

Department of Information Technology, Vidyalankar Institute of Technology, Mumbai (University of Mumbai)

Module overview

#	Module	Purpose
1	**Document Input Module**	Accepts formal documents (PDF, DOCX, XLSX, CSV) uploaded by the user or chosen from four bundled sample reports, and collects the engine settings.
2	**Document Ingestion and Parsing Module**	Detects the file type and parses it into a document tree of sections, paragraphs, tables and pages.
3	**Table Normalisation Module**	Cleans extracted tables, removes duplicate headers and classifies each table so that it can be used correctly.
4	**Canonical Schema Module**	Holds every extracted fact in strongly typed Pydantic models shared by all later stages.
5	**Semantic Extraction Module**	Extracts metrics, series, comparisons and relationships using a rule-based engine and, optionally, the Groq LLM.
6	**Grounding Gateway Module**	Verifies each extracted value, unit and context against the verbatim source text and assigns a confidence score.
7	**Metric Analysis Module**	Creates the Hero KPIs, ranks comparisons and series, and synthesises insights from the verified model.
8	**Visualization Engine Module**	Chooses a suitable chart for each fact from the shape of the data, scores the candidates and renders interactive Plotly charts.
9	**Dashboard Interface Module**	Provides the Streamlit interface that connects all modules and presents the summary, KPI cards, charts, insights and audit tabs.

How the modules connect



Data moves through the modules in numerical order. The Canonical Schema [4] is the shared data format used by Semantic Extraction [5], the Grounding Gateway [6], Metric Analysis [7], Visualization [8] and the Dashboard [9]. The Dashboard Interface [9] also contains the sidebar controls of the Document Input Module [1] and starts the whole pipeline.

01 Document Input Module

DESCRIPTION

Accepts formal documents (PDF, DOCX, XLSX, CSV) uploaded by the user or chosen from four bundled sample reports, and collects the engine settings.

The Streamlit sidebar and main page receive the user's choices. A file uploader accepts PDF, DOCX, XLSX and CSV files; an uploaded file is written to a temporary file and its name is turned into a readable document title. Alternatively the user can quick-load one of four bundled demo reports (corporate financial report, HR and talent review, sales and logistics operations report, academic accreditation report).

The sidebar also holds the optional Groq API key (read from the GROQ_API_KEY environment variable if present) and the model selector (openai/gpt-oss-120b). The active extraction mode is displayed according to whether a key is given, and when no document is selected the page shows a short description of the pipeline instead of running it.

CODE · app.py

Sidebar settings, sample documents and file upload · app.py

```
# Optional Groq Key
groq_key = st.sidebar.text_input(
    "Groq API Key (Optional)",
    type="password",
    value=os.environ.get("GROQ_API_KEY", ""),
    help="Optional: Enter a Groq API key (free at console.groq.com) for reasoning with openai/gpt-oss-120b. The
        system operates fully offline via deterministic parsing if omitted."
)

groq_model = st.sidebar.selectbox(
    "Groq Model",
    options=[
        "openai/gpt-oss-120b"
    ],
    index=0,
    help="Groq inference using openai/gpt-oss-120b."
)

extraction_mode = f"⚡ Groq ({groq_model}) + Grounding Gateway" if groq_key else " Deterministic Heuristic +
    Grounding Gateway"
st.sidebar.info(f"**Active Mode:**\n{extraction_mode}")

st.sidebar.markdown("---")
st.sidebar.subheader(" Quick-Load Pre-Built Tests")

sample_files = {
    " Corporate Financial Report (PDF)": "sample_documents/financial_report.pdf",
    " HR & Talent Review (DOCX)": "sample_documents/hr_report.docx",
    " Sales & Logistics Operations (PDF)": "sample_documents/sales_operations_report.pdf",
    " Academic Accreditation Report (DOCX)": "sample_documents/academic_report.docx"
}

selected_sample = st.sidebar.selectbox("Choose a demo document:", ["-- Select a Sample --"] +
    list(sample_files.keys()))

# Document Input Router
active_file_path = None
active_doc_title = None

st.markdown('<div class="main-header">Intelligent Formal Document → Analytics Dashboard</div>',
    unsafe_allow_html=True)
st.markdown('<div class="sub-header">Domain-Agnostic Extraction, Anti-Hallucination Grounding & Automated
    Interactive Visualizations</div>', unsafe_allow_html=True)

uploaded_file = st.file_uploader(
    "Upload a formal document (PDF, DOCX, XLSX, CSV)",
    type=["pdf", "docx", "xlsx", "csv"],
    help="Provide any organizational report. The pipeline will parse text, tables, and generate an analytical
        dashboard without manual schema mapping."
)

if uploaded_file is not None:
    raw_name = os.path.splitext(uploaded_file.name)[0]
    active_doc_title = re.sub(r'[_\-]+', ' ', raw_name).strip().title()
    suffix = f".{uploaded_file.name.split('.')[-1]}"
    with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
        tmp.write(uploaded_file.read())
```

```
        active_file_path = tmp.name
elif selected_sample != "-- Select a Sample --":
    active_file_path = sample_files[selected_sample]
    active_doc_title = selected_sample.split(" (")[0].replace(" ", "").replace(" ", "").replace(" ",
        "").replace(" ", "").strip()

if not active_file_path:
    ...
```

02 Document Ingestion and Parsing Module

DESCRIPTION

Detects the file type and parses it into a document tree of sections, paragraphs, tables and pages.

DocumentParser is the unified router: it checks that the file exists, selects the PDF, DOCX or spreadsheet parser from the file extension, and raises a clear error for unsupported formats. Every parser returns the same dictionary (title, page count, pages, all tables, full text), so later stages do not depend on the file format.

The PDF parser uses PyMuPDF to detect headings from font size and bold flags, and pdfplumber to extract page text and tables. The DOCX parser walks the document body in natural flow order, so tables stay interleaved with the paragraphs around them, removes empty spacer columns, and estimates virtual page numbers so that every fact can still carry a page citation. XLSX and CSV files are loaded with pandas as a single-page document with one table.

CODE · ingestion/document_parser.py, ingestion/docx_extractor.py, ingestion/pdf_extractor.py

Document router and spreadsheet parser · ingestion/document_parser.py

```
class DocumentParser:

    @classmethod
    def parse(cls, file_path: str, original_title: Optional[str] = None) -> Dict[str, Any]:
        """Parses any supported document format and returns a canonical DocumentTree dictionary."""
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"Document not found: {file_path}")

        ext = os.path.splitext(file_path)[1].lower()

        if ext == ".pdf":
            res = PDFExtractor.extract(file_path)
        elif ext in [".docx", ".doc"]:
            res = DOCXExtractor.extract(file_path)
        elif ext in [".xlsx", ".xls", ".csv"]:
            res = cls._parse_spreadsheet(file_path, ext)
        else:
            raise ValueError(f"Unsupported file format '{ext}'. Supported formats: PDF, DOCX, XLSX, CSV.")

        if original_title:
            res["title"] = original_title
        elif re.search(r'^(?:tmp[a-z0-9_]+|temp[a-z0-9_]+)$', res.get("title", ""), re.IGNORECASE):
            # Attempt to find prominent heading from first page
            found_title = None
            if res.get("pages"):
                first_page = res["pages"][0]
                for p in getattr(first_page, "paragraphs", []):
                    if getattr(p, "is_heading", False) and len(p.text) > 3:
                        found_title = p.text.strip()
                        break
            res["title"] = found_title or "Executive Formal Report"

        return res

    @classmethod
    def _parse_spreadsheet(cls, file_path: str, ext: str) -> Dict[str, Any]:
        """Parses tabular spreadsheets as a single-page document with tables."""
        doc_title = os.path.basename(file_path).rsplit(".", 1)[0].replace("_", " ").title()
        if ext == ".csv":
            df = pd.read_csv(file_path)
        else:
            df = pd.read_excel(file_path)

        headers = [str(c) for c in df.columns]
        rows = [[str(val) if pd.notna(val) else "" for val in row] for row in df.values]

        table = RawTable(
            headers=headers,
            rows=rows,
            page_number=1,
            section_title="Data Sheet",
            table_index=0
        )

        para = RawParagraph(
            text=f"Spreadsheet dataset containing {len(df)} records across columns: {', '.join(headers)}.",
            page_number=1,
            section_title="Data Sheet",
            is_heading=False
```

```
)

page = DocumentPage(
    page_number=1,
    text=table.to_markdown(),
    paragraphs=[para],
    tables=[table]
)

return {
    "title": doc_title,
    "file_path": file_path,
    "file_type": ext.lstrip("."),
    "page_count": 1,
    "pages": [page],
    "all_tables": [table],
    "full_text": table.to_markdown()
}
```

PDF heading detection and table extraction · ingestion/pdf_extractor.py

```
# 1. Use PyMuPDF for font-based heading detection
doc_fitz = pymupdf.open(file_path)
page_headings: Dict[int, List[Dict[str, Any]]] = {}

for p_no in range(len(doc_fitz)):
    fitz_page = doc_fitz[p_no]
    blocks = fitz_page.get_text("dict")["blocks"]
    headings = []

    for b in blocks:
        if b.get("type") == 0:  # text block
            for line in b.get("lines", []):
                for span in line.get("spans", []):
                    txt = span.get("text", "").strip()
                    sz = span.get("size", 10.0)
                    flags = span.get("flags", 0)  # 2 is italic, 16 is bold
                    if len(txt) > 3 and (sz >= 12.0 or flags & 16):
                        headings.append({
                            "text": txt,
                            "size": sz,
                            "is_bold": bool(flags & 16),
                            "bbox": span.get("bbox")
                        })
    page_headings[p_no + 1] = headings
doc_fitz.close()

# 2. Use pdfplumber for table extraction and structured page text
with pdfplumber.open(file_path) as pdf:
    current_section = ""
    table_global_idx = 0

    for p_idx, page in enumerate(pdf.pages):
        p_num = p_idx + 1
        page_text = page.extract_text() or ""
        full_text_list.append(page_text)

        # Extract tables on this page
        raw_extracted_tables = page.extract_tables()
        page_tables: List[RawTable] = []

        for t in raw_extracted_tables:
            if not t or len(t) < 2:
                continue
            # Clean empty rows
            cleaned_t = [[cell if cell is not None else "" for cell in row] for row in t]
            header = cleaned_t[0]
            data_rows = cleaned_t[1:]
            table_obj = RawTable(
                headers=header,
                rows=data_rows,
                page_number=p_num,
                section_title=current_section,
                table_index=table_global_idx
            )
            page_tables.append(table_obj)
            all_tables.append(table_obj)
            table_global_idx += 1
```

DOCX extraction in document flow order · ingestion/docx_extractor.py

```
# Traverse body elements in natural document flow order
for child in doc.element.body:
    if child.tag.endswith('p'):
        p = Paragraph(child, doc)
        text = p.text.replace('\xa0', ' ').replace('\u202f', ' ').strip()
        if not text:
            continue
```

```
            # Check for page break elements in runs
            has_page_break = False
            for r in p.runs:
                xml = r._r.xml
                if 'w:br w:type="page"' in xml or 'w:lastRenderedPageBreak' in xml:
                    has_page_break = True
                    break

            style_name = p.style.name.lower() if p.style else ""

            # Determine heading status
            is_heading = "heading" in style_name or "title" in style_name
            heading_lvl = 1 if "1" in style_name else (2 if "2" in style_name else 0)

            # Heading fallback: short bold runs or uppercase
            if not is_heading and len(text) < 80:
                runs = [r for r in p.runs if r.text.strip()]
                if runs and all(r.bold for r in runs):
                    is_heading = True
                    heading_lvl = 2
                elif text.isupper() and len(text.split()) <= 6:
                    is_heading = True
                    heading_lvl = 2

            if "title" in style_name:
                doc_title = text
                is_heading = True
                heading_lvl = 1
            elif is_heading:
                current_section = text

            para_obj = RawParagraph(
                text=text,
                page_number=current_page_no,
                section_title=current_section,
                is_heading=is_heading,
                heading_level=heading_lvl
            )
            current_page_paras.append(para_obj)
            all_paragraphs.append(para_obj)
            current_page_text.append(text)
            current_page_chars += len(text)

            if has_page_break or current_page_chars >= PAGE_CHAR_THRESHOLD:
                commit_page()

        elif child.tag.endswith('tbl'):
            t = Table(child, doc)
            if not t.rows or len(t.rows) < 2:
                continue

            # Extract all row cell texts
            raw_grid: List[List[str]] = []
            for r in t.rows:
                raw_grid.append([c.text.strip() for c in r.cells])

            if not raw_grid or not raw_grid[0]:
                continue

            num_cols = len(raw_grid[0])
            # Prune columns that are completely empty across all rows (spacer columns)
            active_col_indices = [
                col_idx for col_idx in range(num_cols)
                if any(row[col_idx].strip() for row in raw_grid if col_idx < len(row))
            ]

            if not active_col_indices:
                continue

            cleaned_headers = [raw_grid[0][col_idx] for col_idx in active_col_indices]
            cleaned_rows = [
                [row[col_idx] if col_idx < len(row) else "" for col_idx in active_col_indices]
                for row in raw_grid[1:]
            ]

            table_obj = RawTable(
                headers=cleaned_headers,
                rows=cleaned_rows,
                page_number=current_page_no,
                section_title=current_section,
                table_index=table_idx
            )
            current_page_tables.append(table_obj)
            all_tables.append(table_obj)
            table_idx += 1
            md_repr = table_obj.to_markdown()
            current_page_text.append(md_repr)
            current_page_chars += len(md_repr)
            if current_page_chars >= PAGE_CHAR_THRESHOLD:
```

```
commit_page()
```

03 Table Normalisation Module

DESCRIPTION

Cleans extracted tables, removes duplicate headers and classifies each table so that it can be used correctly.

Raw tables from PDF and Word files contain line breaks, control characters, wrapped headers and repeated column names. This module cleans every cell, gives empty or repeated headers unique names (Col_1, Revenue_2 and so on), repairs fragmented headers, and parses numeric cells such as 164,600, 96.2%, (1,234) or an amount in Crore or Lakh into a value, a unit and an optional signed change.

TableNormalizer then repairs headers that continue into the first data row, classifies the table as a performance comparison (periods in columns), a categorical breakdown (segments, departments, regions) or a non-analytical schedule, infers the dimension name, and detects the scale unit (Million, Billion, Crore, Lakh) from the headers.

CODE · ingestion/pdf_extractor.py, ingestion/table_normalizer.py

Cell cleaning and table object · ingestion/pdf_extractor.py

```
def clean_cell_text(text: Optional[str]) -> str:
    """Cleans raw PDF/table cell text, fixing line breaks, artifacts, and multi-spaces."""
    if not text:
        return ""
    # Normalize unicode spaces and clean newlines
    cleaned = text.replace("\n", " ").replace("\r", " ")
    # Replace common font encoding quirks for currency symbols
    cleaned = re.sub(r'[\x00-\x1f\x7f-\x9f]', '', cleaned)
    cleaned = re.sub(r'\s+', ' ', cleaned).strip()
    return cleaned


class RawTable:
    def __init__(self, headers: List[str], rows: List[List[str]], page_number: int, section_title: str = "",
        table_index: int = 0):
        from ingestion.table_normalizer import deduplicate_headers
        clean_hdrs = [clean_cell_text(h) if clean_cell_text(h) else f"Col_{i}" for i, h in enumerate(headers)]
        self.headers = deduplicate_headers(clean_hdrs)
        self.rows = [[clean_cell_text(c) for c in row] for row in rows]
        self.page_number = page_number
        self.section_title = section_title
        self.table_index = table_index

    def to_dict(self) -> Dict[str, Any]:
        return {
            "page_number": self.page_number,
            "section_title": self.section_title,
            "table_index": self.table_index,
            "headers": self.headers,
            "rows": self.rows
        }

    def to_markdown(self) -> str:
        if not self.headers:
            return ""
        hdr_line = "| " + " | ".join(self.headers) + " |"
        sep_line = "| " + " | ".join(["---"] * len(self.headers)) + " |"
        row_lines = ["| " + " | ".join(row) + " |" for row in self.rows]
        return "\n".join([hdr_line, sep_line] + row_lines)
```

Header deduplication, header repair and numeric cell parsing · ingestion/table_normalizer.py

```
def deduplicate_headers(headers: List[str]) -> List[str]:
    """Ensures all column headers are unique, non-empty strings suitable for Pandas and PyArrow."""
    seen: Dict[str, int] = {}
    deduped: List[str] = []
    for i, h in enumerate(headers):
        clean_name = str(h).strip() if h is not None and str(h).strip() else f"Col_{i+1}"
        if clean_name in seen:
            seen[clean_name] += 1
            deduped.append(f"{clean_name}_{seen[clean_name]}")
        else:
            seen[clean_name] = 1
            deduped.append(clean_name)
    return deduped
def clean_header_name(header: str) -> str:
    """Repairs fragmented header names caused by table wrapping."""
```

```
    if not header:
        return ""
    h = header.replace("\n", " ").strip()
    h = re.sub(r'\s+', ' ', h)
    # Fix broken currency / parenthesis artifacts
    h = re.sub(r'\(INR\s*C\s*r\)', '(INR Cr)', h)
    h = re.sub(r'\(n\s*Cr\)', '(INR Cr)', h)
    h = re.sub(r'\(n\s*CQr\)', '(INR Cr)', h)
    h = re.sub(r'^\)\s*', '', h)
    h = re.sub(r'\s*\(?Q$', '', h)
    return h.strip()


def parse_numeric_cell(cell_str: str) -> Tuple[Optional[float], str, Optional[float]]:
    """
    Extracts numeric value, unit, and optional sign/percentage from a table cell.
    Returns: (numeric_value, unit, percentage_change_if_signed)
    """
    if not cell_str:
        return None, "", None

    s = cell_str.strip()

    # Check for growth/change signed pattern e.g. "+21.1%", "-5.6%", "15%"
    signed_pct_match = re.search(r'^([+-]?\d+(?:\.\d+)?)\s*%$', s)
    if signed_pct_match:
        val = float(signed_pct_match.group(1))
        return val, "%", val

    # Check standard percentage e.g. "96.2%"
    pct_match = re.search(r'(\d+(?:\.\d+)?)\s*%', s)
    if pct_match:
        val = float(pct_match.group(1))
        return val, "%", None

    # Check standard comma-separated or integer/float e.g. "164,600", "281,724", "13.64"
    # Handle currency symbols and parentheses for negative values: (1,234) -> -1234
    is_negative = False
    if re.search(r'^\s*\(([\d,.]+)\)\s*$', s):
        is_negative = True
        s = re.sub(r'^\s*\(([\d,.]+)\)\s*$', r'\1', s)

    clean_num_str = s.replace(",", "").replace("$", "").replace("₹", "").replace("€", "").strip()
    num_match = re.search(r'[-+]?\d+(?:\.\d+)?', clean_num_str)
    if num_match:
        try:
            val = float(num_match.group(0))
            if is_negative:
                val = -val
            # Detect unit suffix if any
            unit = ""
            if "cr" in s.lower():
                unit = "Crore"
            elif "lakh" in s.lower() or "lpa" in s.lower():
                unit = "Lakh"
            elif "%" in s:
                unit = "%"
            return val, unit, None
        except ValueError:
            pass

    return None, "", None
```

Table topology types and normaliser · ingestion/table_normalizer.py

```
class TableTopology(str, Enum):
    PERFORMANCE_COMPARISON = "performance_comparison"
    CATEGORICAL_BREAKDOWN = "categorical_breakdown"
    NON_ANALYTICAL_SCHEDULE = "non_analytical_schedule"


class TableNormalizer:

    @staticmethod
    def normalize_table(headers: List[str], rows: List[List[str]], section_title: str = "") -> Dict[str, Any]:
        """
        Analyzes table structure:
        - Repaired column names
        - Topology classification (Performance Comparison vs Categorical Breakdown vs Schedule)
        - Contextual dimension name inference
        - Parsed metric columns and clean rows
        """
        cleaned_headers = deduplicate_headers([clean_header_name(h) for h in headers])

        # Check if row 0 was a continuation of headers
        if rows and len(rows) > 1:
            first_row_numeric_count = sum(1 for cell in rows[0] if parse_numeric_cell(cell)[0] is not None)
            if first_row_numeric_count == 0 and len(rows[0]) == len(cleaned_headers):
```

```
        combined = []
        for h, sub in zip(cleaned_headers, rows[0]):
            combined.append(f"{h} {sub}".strip())
        cleaned_headers = deduplicate_headers([clean_header_name(c) for c in combined])
        rows = rows[1:]

topology = classify_table_topology(cleaned_headers, rows, section_title)

dim_raw = cleaned_headers[0] if cleaned_headers else "Category"
dim_name = infer_dimension_name(dim_raw, section_title, cleaned_headers, rows)

# Detect default table unit from scale markers in headers
default_unit = ""
all_header_text = " ".join(headers).lower()
if "billion" in all_header_text:
    default_unit = "Billion"
elif "million" in all_header_text:
    default_unit = "Million"
elif "thousand" in all_header_text:
    default_unit = "Thousand"
elif "crore" in all_header_text:
    default_unit = "Crore"
elif "lakh" in all_header_text:
    default_unit = "Lakh"

metric_columns = []
for col_idx in range(1, len(cleaned_headers)):
    col_name = cleaned_headers[col_idx]
    display_name = re.sub(r'_\d+$', '', col_name).strip()

    # Check how many rows have numeric values in this column
    numeric_cells = []
    for row in rows:
        if col_idx < len(row):
            val, unit, _ = parse_numeric_cell(row[col_idx])
            if val is not None:
                numeric_cells.append((val, unit))

    if len(numeric_cells) >= max(1, len(rows) * 0.4):
        unit = ""
        is_share_pct = ("market share" in col_name.lower() or "revenue share" in col_name.lower() or "share
            of" in col_name.lower()) and not any(s in col_name.lower() for s in ["shares", "per share"])
        if "%" in col_name or "rate" in col_name.lower() or is_share_pct or "change" in col_name.lower():
            unit = "%"
        elif re.search(r'\b(?:per\s+share|eps)\b', col_name.lower()):
            unit = "$ / share" if "$" in col_name or default_unit == "Million" or default_unit == "Billion"
                else "/ share"
        elif re.search(r'\b(?:years?|months?|days?|life|term|tenure)\b', col_name.lower()):
            if "month" in col_name.lower():
                unit = "Months"
            elif "day" in col_name.lower():
                unit = "Days"
            else:
                unit = "Years"
        elif "shares" in col_name.lower():
            unit = f"{default_unit} Shares".strip() if default_unit else "Shares"
        elif "cr" in col_name.lower() or "crore" in col_name.lower():
            unit = "Crore"
        elif "lpa" in col_name.lower() or "lakh" in col_name.lower():
            unit = "Lakh"
        elif "employee" in col_name.lower() or "headcount" in col_name.lower():
            unit = "Employees"
        elif "student" in col_name.lower():
            unit = "Students"
        elif "deliver" in col_name.lower():
            unit = "Deliveries"
        elif "count" in col_name.lower():
            unit = "Units"
        else:
            if numeric_cells and numeric_cells[0][1]:
                unit = numeric_cells[0][1]
            elif default_unit:
                unit = default_unit

        metric_columns.append({
            "col_idx": col_idx,
            "metric_name": display_name,
            "unit": unit
        })

return {
    "topology": topology,
    "dimension_name": dim_name,
    "headers": cleaned_headers,
    "metric_columns": metric_columns,
    "rows": rows,
    "default_unit": default_unit
}
```

04 Canonical Schema Module

DESCRIPTION

Holds every extracted fact in strongly typed Pydantic models shared by all later stages.

The canonical intermediate representation decouples extraction from analysis and visualisation. ProvenanceRecord stores the source document, page number, section title and the verbatim source span of a fact. GroundingStatus stores whether the value, metric context and unit were matched, a confidence score between 0 and 1, and explanatory notes.

StandaloneMetric, ComparisonItem, CategoricalSeries, TimeSeries, Relationship and AnalyticalInsight carry the facts themselves, and DocumentAnalyticsModel collects them for one document together with the inferred domain, reporting period, executive summary and extraction metadata. Because both extraction modes produce this same model, the rest of the pipeline does not need to know which mode was used.

CODE · schema/models.py

Provenance and grounding models · schema/models.py

```
class ProvenanceRecord(BaseModel):
    """Tracks exact provenance to the source document."""
    source_document: str = Field(description="Name or path of the source document")
    page_number: int = Field(default=1, description="1-indexed page number where the fact was found")
    section_title: Optional[str] = Field(default=None, description="Enclosing heading or section title")
    verbatim_source_span: str = Field(description="Exact sentence or table row/cell from the document")
    source_type: str = Field(default="prose", description="'prose' or 'table'")
    table_index: Optional[int] = Field(default=None, description="Index of table if source_type == 'table'")


class GroundingStatus(BaseModel):
    """Grounding verification details ensuring value + metric + unit co-occur in the source span."""
    is_grounded: bool = Field(default=True, description="True if verified against source span")
    value_matched: bool = Field(default=True, description="Numeric value verified in source span")
    metric_context_matched: bool = Field(default=True, description="Metric name tokens found in source span")
    unit_matched: bool = Field(default=True, description="Unit found in or inferred from source span")
    confidence_score: float = Field(default=1.0, ge=0.0, le=1.0, description="Overall confidence [0.0 - 1.0]")
    grounding_notes: str = Field(default="Verified directly against document span.")
```

Metric, series and document models · schema/models.py

```
class StandaloneMetric(BaseModel):
    """Individual high-level quantitative indicator."""
    id: str
    name: str
    current_value: float
    display_value: str = ""
    unit: str = ""
    canonical_value: Optional[float] = None
    previous_value: Optional[float] = None
    change_abs: Optional[float] = None
    change_pct: Optional[float] = None
    comparison_period: Optional[str] = None  # e.g., "YoY", "QoQ", "vs Last Year"
    favorable_direction: str = Field(default="neutral", description="'up', 'down', or 'neutral'")
    period: Optional[str] = None
    provenance: ProvenanceRecord
    grounding: GroundingStatus = Field(default_factory=GroundingStatus)


class CategoricalDataPoint(BaseModel):
    """Single item in a categorical series."""
    label: str
    value: float
    secondary_value: Optional[float] = None  # e.g., for multi-metric tables (SLA and Return Rate)
    share_pct: Optional[float] = None


class CategoricalSeries(BaseModel):
    """Series representing distributions, departmental splits, or regional breakdowns."""
    id: str
    metric_name: str
    dimension_name: str  # e.g., "Department", "Business Segment", "Zone"
    unit: str = ""
    secondary_metric_name: Optional[str] = None
    secondary_unit: Optional[str] = None
    is_part_of_whole: bool = Field(default=False, description="True if items represent composition summing near
        100%")
```

```
    items: List[CategoricalDataPoint] = Field(default_factory=list)
    provenance: ProvenanceRecord
    grounding: GroundingStatus = Field(default_factory=GroundingStatus)


class DocumentAnalyticsModel(BaseModel):
    """The central Intermediate Representation (IR) of the entire document."""
    document_title: str
    inferred_domain: str = Field(default="General", description="e.g. Finance, HR, Operations, Academic")
    reporting_period: Optional[str] = None
    executive_summary: str = ""
    standalone_metrics: List[StandaloneMetric] = Field(default_factory=list)
    comparisons: List[ComparisonItem] = Field(default_factory=list)
    categorical_series: List[CategoricalSeries] = Field(default_factory=list)
    time_series: List[TimeSeries] = Field(default_factory=list)
    relationships: List[Relationship] = Field(default_factory=list)
    insights: List[AnalyticalInsight] = Field(default_factory=list)
    extraction_metadata: Dict[str, Any] = Field(default_factory=dict)
```

05 Semantic Extraction Module

DESCRIPTION

Extracts metrics, series, comparisons and relationships using a rule-based engine and, optionally, the Groq LLM.

SemanticExtractor coordinates extraction. It always runs the deterministic HeuristicExtractor first, which reads every prose sentence and every normalised table with regular expressions and table topology rules, infers the domain and reporting period, removes duplicate metrics and writes an executive summary. This guarantees a result without internet access or an API key.

If a Groq API key is available, the most informative pages of the document are sent to openai/gpt-oss-120b with a strict JSON schema prompt and temperature 0. The prompt requires every number to be stated in the document and every source span to be copied verbatim. The returned entities are merged into the same canonical model, and if the call fails the system falls back to the heuristic result and records the failure status. A Gemini provider is also supported as an alternative. In every case the merged model is passed to the grounding gateway before it is used.

CODE · extraction/heuristic_extractor.py, extraction/semantic_extractor.py

Extraction rules given to the language model · extraction/semantic_extractor.py

```
CRITICAL RULES:
1. Every numeric value MUST be explicitly stated in the document. NEVER invent or extrapolate numbers.
2. The "verbatim_source_span" MUST be an exact copy-paste from the text provided.
3. Preserve complete compound noun phrases (e.g., 'Cash, cash equivalents, and short-term investments', 'Property,
    plant, and equipment', 'Total operating expenses'). NEVER truncate noun phrases at commas.
4. Focus on primary organizational metrics from Executive Summary, Overview, and Results sections. Avoid extracting
    incidental footnote or accounting disclosures as primary standalone metrics.
5. If no clear previous value or delta is stated, leave previous_value and change_pct as null.
6. Output ONLY raw JSON, with no markdown backticks or commentary.
"""
```

Orchestrator and Groq call · extraction/semantic_extractor.py

```
class SemanticExtractor:

    @classmethod
    def extract(
        cls, doc_tree: Dict[str, Any], api_key: Optional[str] = None,
        provider: str = "groq", model_name: str = "openai/gpt-oss-120b"
    ) -> DocumentAnalyticsModel:
        """
        Executes semantic extraction:
        1. Always extracts deterministic tabular and linguistic facts.
        2. If LLM is available (Groq or Gemini), augments with semantic prose reasoning.
        3. Enforces anti-hallucination grounding verification.
        """
        # Step 1: Base heuristic extraction (guarantees zero failure)
        base_model = HeuristicExtractor.extract(doc_tree)

        # Step 2: Attempt Groq or Gemini LLM augmentation
        groq_key = api_key if provider == "groq" else (os.environ.get("GROQ_API_KEY") or api_key)
        gemini_key = api_key if provider == "gemini" else (os.environ.get("GEMINI_API_KEY") or
            os.environ.get("GOOGLE_API_KEY"))

        if groq_key:
            try:
                base_model = cls._augment_with_groq(doc_tree, base_model, groq_key, model_name=model_name)
                base_model.extraction_metadata["llm_status"] = "success"
                base_model.extraction_metadata["llm_provider"] = "groq"
            except Exception as e:
                logger.warning(f"Groq extraction failed: {e}. Falling back to deterministic model.")
                base_model.extraction_metadata["llm_status"] = f"failed: {e}"
        elif gemini_key:
            try:
                base_model = cls._augment_with_gemini(doc_tree, base_model, gemini_key)
                base_model.extraction_metadata["llm_status"] = "success"
                base_model.extraction_metadata["llm_provider"] = "gemini"
            except Exception as e:
                logger.warning(f"Gemini extraction failed: {e}. Falling back to deterministic model.")
                base_model.extraction_metadata["llm_status"] = f"failed: {e}"
        else:
            base_model.extraction_metadata["llm_status"] = "offline"

        # Step 3: Run Anti-Hallucination & Grounding Validation
```

```
        validated_model = GroundingValidator.validate_and_filter(
            model=base_model,
            doc_text=doc_tree.get("full_text", ""),
            doc_tables=doc_tree.get("all_tables", [])
        )
        return validated_model

    @classmethod
    def _augment_with_groq(
        cls, doc_tree: Dict[str, Any], base_model: DocumentAnalyticsModel,
        api_key: str, model_name: str = "openai/gpt-oss-120b"
    ) -> DocumentAnalyticsModel:
        """Invokes Groq API with strict JSON mode and safe TPM window."""
        from groq import Groq
        client = Groq(api_key=api_key)

        prompt_text = f"Document Text:\n{select_dense_document_text(doc_tree, max_chars=14000)}"
        target_model = model_name or "openai/gpt-oss-120b"

        completion = client.chat.completions.create(
            model=target_model,
            messages=[
                {"role": "system", "content": EXTRACTION_SYSTEM_PROMPT},
                {"role": "user", "content": prompt_text}
            ],
            response_format={"type": "json_object"},
            temperature=0.0
        )

        base_model.extraction_metadata["llm_model"] = target_model
        raw_json = completion.choices[0].message.content.strip()
        # Clean any markdown code fence wrappers
        if raw_json.startswith("```"):
            raw_json = re.sub(r'^```(?:json)?\s*', '', raw_json)
            raw_json = re.sub(r'\s*```$', '', raw_json)

        parsed = json.loads(raw_json)
        return cls._merge_parsed_json(parsed, doc_tree, base_model)
```

Offline heuristic extraction · extraction/heuristic_extractor.py

```
class HeuristicExtractor:

    @classmethod
    def extract(cls, doc_tree: Dict[str, Any], model_name: Optional[str] = None) -> DocumentAnalyticsModel:
        """Extracts complete analytical model deterministically from document tree."""
        doc_title = doc_tree.get("title", "Organizational Report")
        file_path = doc_tree.get("file_path", "")
        doc_name = file_path.split("/")[-1].split("\\")[-1] if file_path else "document"
        pages = doc_tree.get("pages", [])
        all_tables = doc_tree.get("all_tables", [])
        full_text = doc_tree.get("full_text", "")
        total_pages = max(len(pages), 1)

        # 1. Domain & Reporting Period Inference
        inferred_domain = cls._infer_domain(full_text)
        reporting_period = cls._infer_reporting_period(full_text)

        # Sanitize temp file names (e.g. Tmp8g0ltbby) to meaningful titles
        if re.search(r'^(?:tmp[a-z0-9_]+|temp[a-z0-9_]+)$', doc_title, re.IGNORECASE):
            found_title = None
            for p in pages[:2]:
                for para in getattr(p, "paragraphs", []):
                    if getattr(para, "is_heading", False) and len(para.text) > 3 and not
                        re.search(r'^(?:table|page|\d+)', para.text, re.IGNORECASE):
                        found_title = para.text.strip().title()
                        break
                if found_title:
                    break
            doc_title = found_title or f"{inferred_domain} Performance Report"

        standalone_metrics: List[StandaloneMetric] = []
        comparisons: List[ComparisonItem] = []
        categorical_series: List[CategoricalSeries] = []
        relationships: List[Relationship] = []
        insights: List[AnalyticalInsight] = []

        # 2. Extract from Prose Paragraphs
        for page in pages:
            p_num = page.page_number
            rel_pos = p_num / total_pages
            for para in page.paragraphs:
                if para.is_heading or not para.text:
                    continue
                sentences = re.split(r'(?<=[.!?])\s+', para.text)
                for s in sentences:
                    s_clean = s.strip()
                    if not s_clean or len(s_clean) < 10:
```

```
                    continue
                cls._extract_from_sentence(
                    s_clean, p_num, rel_pos, para.section_title, doc_name,
                    standalone_metrics, comparisons, categorical_series, relationships, insights
                )

    # 3. Extract from Tables
    for t_idx, raw_tbl in enumerate(all_tables):
        cls._extract_from_table(
            raw_tbl, t_idx, doc_name,
            standalone_metrics, comparisons, categorical_series
        )

    # 4. Deduplicate Metrics and Comparisons
    deduped_metrics = cls._deduplicate_metrics(standalone_metrics)
    deduped_comparisons = cls._deduplicate_comparisons(comparisons)

    # 5. Generate Executive Summary if empty
    summary = cls._generate_summary(doc_title, inferred_domain, reporting_period, deduped_metrics,
        categorical_series)

    model = DocumentAnalyticsModel(
        document_title=doc_title,
        inferred_domain=inferred_domain,
        reporting_period=reporting_period,
        executive_summary=summary,
        standalone_metrics=deduped_metrics,
        comparisons=deduped_comparisons,
        categorical_series=categorical_series,
        time_series=[],
        relationships=relationships,
        insights=insights,
        extraction_metadata={"method": "deterministic_heuristic", "tables_processed": len(all_tables)}
    )
    return model

@classmethod
def _infer_domain(cls, text: str) -> str:
    t = text.lower()
    if any(w in t for w in ["revenue", "ebitda", "profit", "debt", "financial", "capex", "balance sheet"]):
        return "Financial"
    elif any(w in t for w in ["headcount", "attrition", "time-to-hire", "diversity", "enps", "workforce"]):
        return "Human Resources"
    elif any(w in t for w in ["logistics", "sla", "deliveries", "return rate", "supply chain", "transit"]):
        return "Sales & Operations"
    elif any(w in t for w in ["enrollment", "placement", "academic", "faculty", "curriculum", "scopus", "gpa",
        "ctc"]):
        return "Academic"
    return "Organizational Management"

@classmethod
def _infer_reporting_period(cls, text: str) -> Optional[str]:
    match = re.search(r'(Q[1-4]\s*(?:FY)?\d{2,4}|FY\s*\d{2,4}|Academic
        Year\s*\d{4}-\d{2,4}|AY\s*\d{4}-\d{2,4}|\b20\d{2}\b)', text, re.IGNORECASE)
    if match:
        return match.group(0).strip()
    return None
```

06 Grounding Gateway Module

DESCRIPTION

Verifies each extracted value, unit and context against the verbatim source text and assigns a confidence score.

The grounding gateway is the anti-hallucination step. For each extracted fact it checks that the claimed source span really exists in the document text or table cells, that the numeric value (in plain, decimal or comma-separated form) appears in that span, that at least one key word of the metric name appears in the span, and that the unit is present. When a previous value and a percentage change are both stated, it also recomputes the change and notes any difference.

A fact whose span and value are confirmed receives confidence 1.0 (or 0.7 if the context or arithmetic check fails). A fact that cannot be confirmed receives 0.3, and facts below 0.5 are removed before analysis. The result is stored in the GroundingStatus of each fact, which the dashboard shows as the Verified badge, confidence and page citation.

CODE · schema/validator.py

Number matching and span verification · schema/validator.py

```
def find_number_in_text(val: float, text: str) -> bool:
    """Checks if a float/int number is represented in text (e.g., 842, 842.0, 164,600)."""
    # Try exact int/float string representations
    if str(int(val)) in text or f"{val:.1f}" in text or f"{val:.2f}" in text:
        return True

    # Try comma formatted e.g. 164,600
    try:
        int_val = int(val)
        comma_rep = f"{int_val:,}"
        if comma_rep in text:
            return True
    except (ValueError, OverflowError):
        pass

    # Check rounded / fuzzy numeric match in extracted tokens
    nums = re.findall(r'[-+]?\d+(?:,\d+)*(?:\.\d+)?', text)
    for n in nums:
        clean_n = n.replace(",", "")
        try:
            if abs(float(clean_n) - val) < 0.05:
                return True
        except ValueError:
            pass

    return False


class GroundingValidator:

    @classmethod
    def verify_provenance_span(cls, span: str, document_text: str, document_tables: List[Any]) -> bool:
        """Verifies if the claimed source span exists in the document text or table cells."""
        if not span or len(span.strip()) < 3:
            return False

        clean_span = " ".join(span.split()).lower()
        clean_doc = " ".join(document_text.split()).lower()

        # Check direct substring
        if clean_span in clean_doc:
            return True

        # Check overlap of at least 80% tokens if whitespace/punctuation differed
        span_tokens = normalize_tokens(clean_span)
        if len(span_tokens) >= 4:
            # Check 4-gram sliding window
            for i in range(len(span_tokens) - 3):
                gram = " ".join(span_tokens[i:i+4])
                if gram in clean_doc:
                    return True

        # Check tables
        for t in document_tables:
            for row in t.rows:
                row_str = " ".join(row).lower()
```

```
                if clean_span in row_str or any(word in row_str for word in span_tokens if len(word) > 4):
                    return True

        return False
```

Metric grounding and filtering · schema/validator.py

```
class GroundingValidator:

    @classmethod
    def verify_metric_grounding(cls, metric: StandaloneMetric, doc_text: str, doc_tables: List[Any]) ->
        StandaloneMetric:
        """Verifies standalone metric against its provenance span and document content."""
        span = metric.provenance.verbatim_source_span
        span_exists = cls.verify_provenance_span(span, doc_text, doc_tables)

        val_found = find_number_in_text(metric.current_value, span)
        if not val_found and span_exists:
            # Check if number appears in full doc text near span
            val_found = find_number_in_text(metric.current_value, doc_text)

        # Check context tokens (at least one key word from metric name must be in the span)
        metric_words = [w for w in normalize_tokens(metric.name) if len(w) > 3 and w not in ["rate", "total",
            "ratio", "level"]]
        context_found = any(w in span.lower() for w in metric_words) if metric_words else True

        # Check unit
        unit_words = [w for w in normalize_tokens(metric.unit) if len(w) > 1]
        unit_found = any(w in span.lower() for w in unit_words) if unit_words else True

        # Arithmetic check if both previous and delta are present
        math_ok = True
        notes = []
        if metric.previous_value is not None and metric.change_pct is not None and metric.previous_value != 0:
            calc_pct = ((metric.current_value - metric.previous_value) / abs(metric.previous_value)) * 100.0
            if abs(calc_pct - metric.change_pct) > 1.5:
                # Discrepancy detected
                notes.append(f"Stated delta {metric.change_pct}% differs from computed {calc_pct:.1f}%.")
                math_ok = False

        is_grounded = span_exists and val_found
        conf = 1.0 if (is_grounded and context_found and math_ok) else (0.7 if is_grounded else 0.3)

        if not is_grounded:
            notes.append("Warning: Could not verify numeric value co-occurrence in document span.")
        else:
            notes.append("Verified: Metric, numeric value, and context grounded in source text.")

        metric.grounding = GroundingStatus(
            is_grounded=is_grounded,
            value_matched=val_found,
            metric_context_matched=context_found,
            unit_matched=unit_found,
            confidence_score=conf,
            grounding_notes=" ".join(notes)
        )
        return metric

    @classmethod
    def validate_and_filter(cls, model: DocumentAnalyticsModel, doc_text: str, doc_tables: List[Any]) ->
        DocumentAnalyticsModel:
        """Runs grounding verification on all model components and filters out completely ungrounded
            hallucinations."""
        # 1. Standalone metrics
        verified_metrics = []
        for m in model.standalone_metrics:
            verified_m = cls.verify_metric_grounding(m, doc_text, doc_tables)
            # Retain only grounded items with confidence >= 0.5
            if verified_m.grounding.confidence_score >= 0.5:
                verified_metrics.append(verified_m)
        model.standalone_metrics = verified_metrics

        # 2. Comparisons
        verified_comps = []
        for c in model.comparisons:
            verified_c = cls.verify_comparison_grounding(c, doc_text, doc_tables)
            if verified_c.grounding.confidence_score >= 0.5:
                verified_comps.append(verified_c)
        model.comparisons = verified_comps

        # 3. Categorical series
        verified_series = []
        for s in model.categorical_series:
            verified_s = cls.verify_series_grounding(s, doc_text, doc_tables)
            if verified_s.grounding.confidence_score >= 0.5:
                verified_series.append(verified_s)
        model.categorical_series = verified_series
        return model
```

07 Metric Analysis Module

DESCRIPTION

Creates the Hero KPIs, ranks comparisons and series, and synthesises insights from the verified model.

MetricAnalyzer takes the validated model and prepares what the dashboard displays. It cleans metric and dimension names, replaces generic dimension labels with the section context (Segment, Department, Region), sorts comparisons by size of change, and ranks categorical series that suit a chart (two to six items, part of a whole).

The four Hero KPIs are chosen by a multi-factor score: the importance of the section in which the figure appears, whether it is an aggregate measure rather than a component, whether it has a stated change or previous value, whether it comes from prose, its page position and its grounding confidence. If the document or the LLM supplied no insights, the module generates them from metrics with changes and from the leading item of each series.

CODE · analytics/metric_analyzer.py

Analysis entry point and insight generation · analytics/metric_analyzer.py

```
class MetricAnalyzer:

    @classmethod
    def analyze(cls, model: DocumentAnalyticsModel) -> Dict[str, Any]:
        """
        Analyzes the validated document model:
        1. Ranks standalone metrics for the KPI ribbon using semantic signals (section importance, aggregate scope,
            deltas)
        2. Cleans metric and dimension names, removing generic placeholders
        3. Identifies key comparison pairs
        4. Synthesizes high-level analytical takeaways
        """
        # Clean and sanitize metric names and dimensions
        for m in model.standalone_metrics:
            m.name = clean_metric_name(m.name)

        for c in model.comparisons:
            c.metric_name = clean_metric_name(c.metric_name)

        for s in model.categorical_series:
            s.metric_name = clean_metric_name(s.metric_name)
            if re.search(r'^\(?(?:in\s+)?(?:millions?|billions?|thousands?|crores?|lakhs?|usd|inr|\$)\)?$',
                s.dimension_name, re.IGNORECASE) or s.dimension_name.lower().startswith("col_"):
                sec = s.provenance.section_title or ""
                if "segment" in sec.lower():
                    s.dimension_name = "Segment"
                elif "department" in sec.lower() or "workforce" in sec.lower():
                    s.dimension_name = "Department"
                elif "region" in sec.lower() or "zone" in sec.lower():
                    s.dimension_name = "Region"
                else:
                    s.dimension_name = "Category"

        # 1. Rank metrics for KPI Ribbon using multi-factor semantic prioritization
        ranked_metrics = cls._rank_kpi_metrics(model.standalone_metrics)

        # 2. Key comparisons
        ranked_comparisons = sorted(
            model.comparisons,
            key=lambda c: abs(c.delta_pct or 0.0) if c.delta_pct is not None else abs(c.delta_abs or 0.0),
            reverse=True
        )

        # 3. Filter and rank categorical series
        # Prioritize series with clean item counts (2 to 8 items) and part-of-whole composition
        ranked_series = sorted(
            model.categorical_series,
            key=lambda s: (1.2 if s.is_part_of_whole else 1.0) * (1.1 if 2 <= len(s.items) <= 6 else 0.7),
            reverse=True
        )

        # 4. Generate synthesized insights if none present
        auto_insights = list(model.insights)
        if not auto_insights:
            auto_insights = cls._generate_automated_insights(model)

        return {
```

```
            "title": model.document_title,
            "domain": model.inferred_domain,
            "period": model.reporting_period,
            "summary": model.executive_summary,
            "hero_kpis": ranked_metrics[:4],
            "all_metrics": ranked_metrics,
            "comparisons": ranked_comparisons,
            "series": ranked_series,
            "relationships": model.relationships,
            "insights": auto_insights
        }

    @classmethod
    def _generate_automated_insights(cls, model: DocumentAnalyticsModel) -> List[AnalyticalInsight]:
        """Synthesizes high-level analytical takeaways from metrics, deltas, and series."""
        insights: List[AnalyticalInsight] = []

        # Insights from standalone metrics
        for m in model.standalone_metrics[:3]:
            if m.change_pct is not None and m.previous_value is not None:
                direction = "increased" if m.change_pct > 0 else "decreased"
                sentiment = "positive" if (m.change_pct > 0 and m.favorable_direction == "up") or (m.change_pct < 0
                    and m.favorable_direction == "down") else "negative"
                insights.append(AnalyticalInsight(
                    id=f"auto_ins_{len(insights)+1}",
                    headline=f"{m.name} {direction.capitalize()} by {abs(m.change_pct)}%",
                    detail=f"{m.name} moved from {m.previous_value:g} {m.unit} to {m.current_value:g} {m.unit}
                        ({m.comparison_period or 'period'}).".strip(),
                    sentiment=sentiment,
                    related_metric_ids=[m.id],
                    provenance=m.provenance
                ))

        # Insights from categorical series (Top segment)
        for s in model.categorical_series[:2]:
            if s.items:
                sorted_items = sorted(s.items, key=lambda x: x.value, reverse=True)
                top = sorted_items[0]
                insights.append(AnalyticalInsight(
                    id=f"auto_ins_{len(insights)+1}",
                    headline=f"{top.label} Leads {s.metric_name}",
                    detail=f"Leading contribution in {s.metric_name} is {top.label} at {top.value:g}
                        {s.unit}.".strip(),
                    sentiment="neutral",
                    related_metric_ids=[s.id],
                    provenance=s.provenance
                ))

        return insights
```

Hero KPI ranking · analytics/metric_analyzer.py

```
class MetricAnalyzer:

    @classmethod
    def _rank_kpi_metrics(cls, metrics: List[StandaloneMetric]) -> List[StandaloneMetric]:
        """
        Scores and sorts metrics using semantic signals:
        - Section Semantic Importance: Strong weight to Executive Summary, Overview, Results, MD&A
        - Grammatical Role & Discourse Prominence: Main clause subjects, summary sentences
        - Aggregate vs Component Scope: Aggregate organizational measures vs narrow component details
        - Explicit Comparisons & Deltas: YoY/QoQ percentage changes
        - Grounding Confidence & Supporting Signals
        - Domain vocabulary is strictly a weak supplementary tie-breaker
        """
        def score_metric(m: StandaloneMetric) -> float:
            score = 0.30
            name_lower = m.name.lower()
            sec_title = (m.provenance.section_title or "").lower()

            # 1. Section Semantic Importance (Weight ~0.35)
            high_importance_sections = [
                "executive summary", "executive overview", "overview", "highlights",
                "key findings", "results of operations", "operating results", "results",
                "management discussion", "financial highlights", "performance summary",
                "business overview", "summary of operations"
            ]
            moderate_importance_sections = [
                "segment results", "operations", "departmental review", "review",
                "performance", "discussion"
            ]
            footnote_sections = [
                "notes to consolidated", "accounting policies", "commitments and contingencies",
                "income taxes", "leases", "pension", "footnote", "subsequent events"
            ]

            if any(sec in sec_title for sec in high_importance_sections):
                score += 0.35
```

```
        elif any(sec in sec_title for sec in moderate_importance_sections):
            score += 0.15
        elif any(sec in sec_title for sec in footnote_sections):
            score -= 0.15

        # 2. Aggregate-vs-Component Scope (Weight ~0.20)
        aggregate_terms = [
            "total", "overall", "net", "operating", "consolidated", "full-year",
            "annual", "comprehensive", "core", "gross", "diluted", "revenue"
        ]
        component_terms = [
            "allowance", "accrued", "deferred", "amortization", "withholding",
            "severance", "unvested", "fair value", "foreign exchange", "par value",
            "restructuring", "impairment"
        ]

        if any(agg in name_lower for agg in aggregate_terms):
            score += 0.20
        elif any(comp in name_lower for comp in component_terms):
            score -= 0.15

        # 3. Explicit Comparisons & Deltas (Weight ~0.15)
        if m.change_pct is not None:
            score += 0.10
        if m.previous_value is not None:
            score += 0.05

        # 4. Grammatical Role & Source Prominence (Weight ~0.10)
        if m.provenance.source_type == "prose":
            score += 0.10

        # 5. Document Position Supporting Signal (Weight ~0.05)
        if m.provenance.page_number <= 3:
            score += 0.05

        # 6. Confidence & Supplementary Domain Tie-Breaker (Weight ~0.05)
        score += m.grounding.confidence_score * 0.03
        key_names = ["revenue", "headcount", "enrollment", "volume", "ebitda", "profit", "placement",
            "attrition", "sla"]
        if any(k in name_lower for k in key_names):
            score += 0.02

        return score

    return sorted(metrics, key=score_metric, reverse=True)
```

08 Visualization Engine Module

DESCRIPTION

Chooses a suitable chart for each fact from the shape of the data, scores the candidates and renders interactive Plotly charts.

The taxonomy maps data shapes to chart types: a part-of-whole series of two to six items becomes a donut chart, a temporal series becomes a line trend, a series with long labels or more than four items becomes a horizontal bar chart, a compact series becomes a vertical bar chart, and a two-value comparison becomes a comparison bar chart. Series with too few or too many items, or with all-zero or identical values, are rejected.

The recommender scores each candidate using the section context, the unit, the number of observations, the dimension and the grounding confidence, applies a quality gate of 0.70, removes near-duplicate candidates and returns the best four. The chart factory then draws each one deterministically with Plotly, using a common palette, hover templates and a change badge on comparison charts, instead of relying on generated code.

CODE · visualization/chart_factory.py, visualization/recommendation.py, visualization/taxonomy.py

Chart types and data-shape classification · visualization/taxonomy.py

```
class ChartType(str, Enum):
    KPI_CARD = "kpi_card"
    DONUT = "donut"
    BAR_HORIZONTAL = "bar_horizontal"
    BAR_VERTICAL = "bar_vertical"
    GROUPED_BAR = "grouped_bar"
    COMPARISON_BAR = "comparison_bar"
    LINE_TREND = "line_trend"


class VisualizationTaxonomy:

    @classmethod
    def classify_series(cls, series: CategoricalSeries) -> Optional[ChartCandidate]:
        """
        Determines the best chart candidate for a categorical series.
        Applies chart-type-specific observation bounds:
        - Distributions require 2 <= N <= 12
        - Rejects zero-variance distributions
        """
        if not series.items:
            return None

        n_items = len(series.items)
        # Distribution observation threshold: 2 to 12 items
        if n_items < 2 or n_items > 12:
            return None

        # Variance check: reject series where all values are zero or identical
        vals = [it.value for it in series.items]
        if all(v == 0.0 for v in vals):
            return None
        if max(vals) == min(vals) and n_items > 3:
            return None

        title = f"{series.metric_name} by {series.dimension_name}"

        # Rule 1: Part of whole (percentages or proportions summing near 100%) and 2-6 items -> Donut
        if series.is_part_of_whole and 2 <= n_items <= 6:
            return ChartCandidate(
                candidate_id=f"chart_{series.id}",
                chart_type=ChartType.DONUT,
                title=title,
                data_source=series,
                metadata={"dimension": series.dimension_name, "metric": series.metric_name, "unit": series.unit}
            )

        # Rule 1.5: Temporal sequence -> Line Trend Chart
        import re
        dim_clean = series.dimension_name.lower().strip()
        is_temporal_dim = bool(re.search(r'\b(year|quarter|month|fiscal|fy|period|trend)\b', dim_clean))
        all_temporal_labels = all(bool(re.search(r'^(?:20\d\d|19\d\d|fy\s*\d\d|q[1-4]|h[1-2])\b',
            it.label.lower().strip())) for it in series.items)
        if (is_temporal_dim or all_temporal_labels) and n_items >= 2:
            return ChartCandidate(
                candidate_id=f"chart_{series.id}",
```

```
                chart_type=ChartType.LINE_TREND,
                title=f"{series.metric_name} Trend over {series.dimension_name}",
                data_source=series,
                metadata={"dimension": series.dimension_name, "metric": series.metric_name, "unit": series.unit}
            )

        # Rule 2: If items have long labels or > 4 items -> Horizontal Bar
        avg_label_len = sum(len(it.label) for it in series.items) / max(1, n_items)
        if avg_label_len > 12 or n_items > 4:
            return ChartCandidate(
                candidate_id=f"chart_{series.id}",
                chart_type=ChartType.BAR_HORIZONTAL,
                title=title,
                data_source=series,
                metadata={"dimension": series.dimension_name, "metric": series.metric_name, "unit": series.unit}
            )

        # Rule 3: Vertical Bar for compact categories (e.g. 2-4 items with short labels)
        return ChartCandidate(
            candidate_id=f"chart_{series.id}",
            chart_type=ChartType.BAR_VERTICAL,
            title=title,
            data_source=series,
            metadata={"dimension": series.dimension_name, "metric": series.metric_name, "unit": series.unit}
        )

    @classmethod
    def classify_comparison(cls, comp: ComparisonItem) -> Optional[ChartCandidate]:
        """
        Creates a side-by-side comparison bar chart candidate.
        A 2-category comparison (e.g. Actual vs Target, Current vs Previous Year) is explicitly valid (N = 2).
        """
        if comp.baseline_value == 0.0 and comp.current_value == 0.0:
            return None

        title = f"{comp.metric_name}: {comp.baseline_label} vs {comp.current_label}"
        return ChartCandidate(
            candidate_id=f"chart_comp_{comp.id}",
            chart_type=ChartType.COMPARISON_BAR,
            title=title,
            data_source=comp,
            metadata={"baseline": comp.baseline_value, "current": comp.current_value, "unit": comp.unit}
        )
```

Candidate scoring, quality gate and ranking · visualization/recommendation.py

```
class VisualizationRecommender:

    @classmethod
    def recommend_charts(
        cls,
        series_list: List[CategoricalSeries],
        comparisons: List[ComparisonItem],
        max_charts: int = 4
    ) -> List[ChartCandidate]:
        """Generates, scores, filters by quality gate, and ranks candidate charts."""
        candidates: List[ChartCandidate] = []

        # 1. Generate candidates from categorical series (distributions, segments)
        for s in series_list:
            if not s.items:
                continue
            cand = VisualizationTaxonomy.classify_series(s)
            if cand is not None:
                cand.score = cls._score_series_candidate(s, cand.chart_type)
                # Tightened Quality Gate: omit candidates scoring below 0.70
                if cand.score >= 0.70:
                    candidates.append(cand)

        # 2. Generate candidates from explicit comparisons (YoY / Target vs Actual)
        for c in comparisons:
            cand = VisualizationTaxonomy.classify_comparison(c)
            if cand is not None:
                cand.score = cls._score_comparison_candidate(c)
                # Tightened Quality Gate: omit candidates scoring below 0.70
                if cand.score >= 0.70:
                    candidates.append(cand)

        # 3. Deduplicate Redundant Candidates
        filtered = cls._filter_redundancy(candidates)

        # 4. Sort by score descending and take top N
        sorted_charts = sorted(filtered, key=lambda c: c.score, reverse=True)
        return sorted_charts[:max_charts]

    @classmethod
    def _score_comparison_candidate(cls, c: ComparisonItem) -> float:
        sec_lower = (c.provenance.section_title or "").lower().strip()
```

```
        metric_lower = c.metric_name.lower()
        score = 0.65

        # 1. Section context
        if any(w in sec_lower for w in ["executive", "overview", "highlight", "results", "operating",
            "performance", "segment", "discussion"]):
            score += 0.20
        elif any(w in sec_lower for w in ["note", "disclosure", "accounting policy", "commitments", "footnote",
            "statutory"]):
            score -= 0.25

        # 2. Meaningful delta percentage bonus
        if c.delta_pct is not None:
            score += min(0.20, abs(c.delta_pct) / 50.0)

        score += c.grounding.confidence_score * 0.10

        # 3. Primary operational indicators
        key_indicators = ["revenue", "profit", "income", "margin", "volume", "deliveries", "headcount",
            "placement", "ebitda", "sales", "enrollment"]
        if any(k in metric_lower for k in key_indicators):
            score += 0.10

        return round(score, 3)
```

Plotly chart factory · visualization/chart_factory.py

```
def format_val(val: float, unit: str = "") -> str:
    """Formats numeric values with clean abbreviations, thousand separators, and units."""
    if val is None:
        return ""
    try:
        fval = float(val)
    except (ValueError, TypeError):
        return str(val)

    if abs(fval) >= 1_000_000:
        val_str = f"{fval/1_000_000:.2f}".rstrip("0").rstrip(".") + "M"
    elif abs(fval) >= 10_000:
        val_str = f"{fval:,.0f}"
    elif fval.is_integer():
        val_str = f"{int(fval):,}"
    else:
        val_str = f"{fval:.1f}"

    clean_unit = str(unit or "").strip()
    if clean_unit and clean_unit != "%":
        return f"{val_str} {clean_unit}".strip()
    elif clean_unit == "%":
        return f"{val_str}%"
    return val_str


class ChartFactory:

    @classmethod
    def render_chart(cls, candidate: ChartCandidate) -> go.Figure:
        """Entry point that renders a Plotly Figure from a ChartCandidate."""
        if candidate.chart_type == ChartType.DONUT:
            return cls.render_donut(candidate.data_source, candidate.title)
        elif candidate.chart_type == ChartType.BAR_HORIZONTAL:
            return cls.render_horizontal_bar(candidate.data_source, candidate.title)
        elif candidate.chart_type == ChartType.BAR_VERTICAL:
            return cls.render_vertical_bar(candidate.data_source, candidate.title)
        elif candidate.chart_type == ChartType.COMPARISON_BAR:
            return cls.render_comparison_bar(candidate.data_source, candidate.title)
        elif candidate.chart_type == ChartType.LINE_TREND:
            return cls.render_line_trend(candidate.data_source, candidate.title)
        else:
            return cls.render_vertical_bar(candidate.data_source, candidate.title)

    @classmethod
    def render_comparison_bar(cls, comp: ComparisonItem, title: str) -> go.Figure:
        categories = [comp.baseline_label, comp.current_label]
        values = [comp.baseline_value, comp.current_value]
        is_favorable = (comp.delta_abs or 0) >= 0
        current_bar_color = "#10B981" if is_favorable else "#EF4444"
        bar_colors = ["#94A3B8", current_bar_color]
        unit = comp.unit or ""

        fig = go.Figure(data=[go.Bar(
            x=categories,
            y=values,
            marker=dict(
                color=bar_colors,
                cornerradius=6,
                line=dict(color="rgba(255,255,255,0.7)", width=1.5)
            ),
```

```
    text=[format_val(v, unit) for v in values],
    textposition="outside",
    cliponaxis=False,
    textfont=dict(size=11, color="#334155", family="Inter, sans-serif"),
    hovertemplate=f"<b>%{{x}}</b><br>Value: %{{y}} {unit}<extra></extra>"
)])

cls._apply_base_layout(fig, title, height=350)

# Delta badge annotation in top-right corner
delta_str = ""
if comp.delta_pct is not None:
    arrow = "▲" if comp.delta_pct >= 0 else "▼"
    delta_str = f"{arrow} {comp.delta_pct:+g}%"
elif comp.delta_abs is not None:
    arrow = "▲" if comp.delta_abs >= 0 else "▼"
    delta_str = f"{arrow} {comp.delta_abs:+g} {unit}".strip()

if delta_str:
    badge_color = "#10B981" if is_favorable else "#EF4444"
    bg_color = "rgba(16, 185, 129, 0.14)" if is_favorable else "rgba(239, 68, 68, 0.14)"
    fig.add_annotation(
        text=f"<span style='color:{badge_color}; background:{bg_color}; padding:4px 10px;
            border-radius:12px; font-weight:700; font-size:11px;'>{delta_str}</span>",
        xref="paper", yref="paper",
        x=0.98, y=1.08,
        showarrow=False,
        align="right"
    )

fig.update_layout(
    bargap=0.45,
    yaxis=dict(
        title=dict(text=f"{comp.metric_name} ({unit})" if unit else comp.metric_name, font=dict(size=11,
            color="#64748B")),
        gridcolor="rgba(148, 163, 184, 0.18)",
        gridwidth=1,
        showgrid=True,
        zeroline=False
    ),
    xaxis=dict(
        showgrid=False,
        tickfont=dict(size=12, color="#334155", family="Inter, sans-serif")
    )
)
return fig
```

09 Dashboard Interface Module

DESCRIPTION

Provides the Streamlit interface that connects all modules and presents the summary, KPI cards, charts, insights and audit tabs.

app.py runs the whole pipeline in four steps with progress indicators (ingestion, extraction and grounding, analysis, chart recommendation) and then renders the dashboard: document title with domain and period badges, the executive summary, the Hero KPI ribbon, up to four recommended charts in a two-column grid, key insights and driver relationships.

Every KPI card and chart has an expandable Source Span panel that shows the source document, page, verbatim sentence, grounding badge and confidence score. Below the dashboard three technical audit tabs show the Grounded Facts table, the canonical representation as JSON, and the structural tables found in the document. The interface components are written for both light and dark themes.

CODE · app.py, ui/components.py

Page configuration and pipeline runner · app.py

```
# Streamlit Page Config
st.set_page_config(
    page_title="Intelligent Document Analytics",
    page_icon="",
    layout="wide",
    initial_sidebar_state="expanded"
)

...

def process_document(file_path: str, api_key: str = None, doc_title: str = None, model_name: str =
    "openai/gpt-oss-120b"):
    """Executes the full pipeline: Ingestion -> Extraction -> Grounding -> Analytics -> Visualization."""
    with st.spinner("1/4 Ingesting document and extracting structural elements..."):
        doc_tree = DocumentParser.parse(file_path, original_title=doc_title)

    with st.spinner(f"2/4 Extracting semantic facts via {model_name if api_key else 'deterministic engine'} &
        grounding..."):
        try:
            model = SemanticExtractor.extract(doc_tree, api_key=api_key, model_name=model_name)
        except TypeError:
            model = SemanticExtractor.extract(doc_tree, api_key=api_key)

    with st.spinner("3/4 Analyzing metrics, comparisons, and deriving key takeaways..."):
        analysis = MetricAnalyzer.analyze(model)

    with st.spinner("4/4 Generating and ranking recommended Plotly visualizations..."):
        recommended_charts = VisualizationRecommender.recommend_charts(
            series_list=analysis["series"],
            comparisons=analysis["comparisons"],
            max_charts=4
        )

    return doc_tree, model, analysis, recommended_charts
```

Hero KPI ribbon with Source Span panels · app.py

```
    # 1. Hero KPI Ribbon
    if analysis["hero_kpis"]:
        st.markdown("###  Key Performance Indicators")
        kpi_cols = st.columns(len(analysis["hero_kpis"]))
        for idx, kpi in enumerate(analysis["hero_kpis"]):
            with kpi_cols[idx]:
                st.markdown(render_kpi_card_html(kpi), unsafe_allow_html=True)
                with st.expander(" Source Span"):
                    st.markdown(render_provenance_badge_html(kpi.provenance, kpi.grounding), unsafe_allow_html=True)
        st.markdown("<br>", unsafe_allow_html=True)
```

Technical audit tabs (grounded facts and canonical IR) · app.py

```
    tab_facts, tab_ir, tab_tables = st.tabs([" Grounded Facts Audit", " Canonical IR (JSON Schema)", " Structural
        Tables"])

    with tab_facts:
        facts_records = []
```

```
        for m in model.standalone_metrics:
            facts_records.append({
                "Metric Name": m.name,
                "Value": m.current_value,
                "Unit": m.unit,
                "Prev Value": m.previous_value,
                "Change %": f"{m.change_pct}%" if m.change_pct else "-",
                "Page": m.provenance.page_number,
                "Grounded": "✓ Verified" if m.grounding.is_grounded else "⚠ Unverified",
                "Confidence": f"{int(m.grounding.confidence_score * 100)}%",
                "Verbatim Source Span": m.provenance.verbatim_source_span
            })
        if facts_records:
            try:
                st.dataframe(pd.DataFrame(facts_records), width="stretch")
            except TypeError:
                st.dataframe(pd.DataFrame(facts_records), use_container_width=True)
        else:
            st.info("No standalone metrics captured.")

    with tab_ir:
        st.caption("Pydantic `DocumentAnalyticsModel` serialized to canonical JSON.")
        st.json(model.model_dump())
```

KPI card and provenance badge components · ui/components.py

```
def render_kpi_card_html(metric: StandaloneMetric) -> str:
    """
    Renders a custom HTML KPI card formatted strictly as a single unindented string.
    Zero leading spaces prevents Markdown parsers from triggering <pre><code> blocks.
    Uses theme-adaptive RGBA colors for seamless Dark/Light mode rendering.
    """
    name = html.escape(metric.name)
    val = f"{metric.current_value:g}"
    unit = html.escape(metric.unit)

    delta_html = ""
    if metric.change_pct is not None:
        sign = "+" if metric.change_pct > 0 else ""
        delta_str = f"{sign}{metric.change_pct:g}%"

        if metric.favorable_direction == "up":
            color = "#10B981" if metric.change_pct >= 0 else "#EF4444"
            bg = "rgba(16, 185, 129, 0.18)" if metric.change_pct >= 0 else "rgba(239, 68, 68, 0.18)"
        elif metric.favorable_direction == "down":
            color = "#10B981" if metric.change_pct <= 0 else "#EF4444"
            bg = "rgba(16, 185, 129, 0.18)" if metric.change_pct <= 0 else "rgba(239, 68, 68, 0.18)"
        else:
            color = "#0284C7"
            bg = "rgba(2, 132, 199, 0.18)"

        period = f" {html.escape(metric.comparison_period)}" if metric.comparison_period else ""
        delta_html = f'<div style="display:inline-flex; align-items:center; background:{bg}; color:{color};
            font-weight:600; font-size:12px; padding:3px 8px; border-radius:12px;
            margin-top:4px;">{delta_str}{period}</div>'

    prev_html = ""
    if metric.previous_value is not None:
        prev_html = f'<div style="font-size:11px; opacity:0.75; margin-top:4px;">Prev: {metric.previous_value:g}
            {unit}</div>'

    # STRICTLY NO LEADING SPACES OR NEWLINES INSIDE HTML FRAGMENTS
    card = (
        f'<div style="border:1px solid rgba(128,128,128,0.25); border-radius:12px; padding:16px 18px; box-shadow:0
            1px 3px rgba(0,0,0,0.06); min-height:120px; display:flex; flex-direction:column;
            justify-content:space-between; background:rgba(128,128,128,0.06);">'
        f'<div style="font-size:12px; font-weight:600; opacity:0.75; text-transform:uppercase;
            letter-spacing:0.5px; margin-bottom:4px;">{name}</div>'
        f'<div style="display:flex; align-items:baseline; gap:6px;">'
        f'<span style="font-size:28px; font-weight:700; line-height:1.2;">{val}</span>'
        f'<span style="font-size:14px; font-weight:500; opacity:0.8;">{unit}</span>'
        f'</div>'
        f'<div style="display:flex; justify-content:space-between; align-items:center; margin-top:6px;">'
        f'{delta_html}'
        f'{prev_html}'
        f'</div>'
        f'</div>'
    )
    return card


def render_provenance_badge_html(prov: ProvenanceRecord, grounding: GroundingStatus) -> str:
    """Renders a theme-adaptive provenance audit badge without leading spaces."""
    doc_name = html.escape(prov.source_document)
    page_num = prov.page_number
    span = html.escape(prov.verbatim_source_span)
    conf = int(grounding.confidence_score * 100)
    badge_color = "#10B981" if grounding.is_grounded else "#F59E0B"
```

```
    badge_text = "Verified Grounded" if grounding.is_grounded else "Uncertain Context"

    badge = (
        f'<div style="border:1px solid rgba(128,128,128,0.2); border-radius:8px; padding:12px; margin-top:6px;
            font-size:12px; background:rgba(128,128,128,0.04);">'
        f'<div style="display:flex; justify-content:space-between; margin-bottom:6px;">'
        f'<span><b>Source:</b> {doc_name} (Page {page_num})</span>'
        f'<span style="border:1px solid {badge_color}; color:{badge_color}; padding:2px 8px; border-radius:10px;
            font-weight:600;">{badge_text} ({conf}%)</span>'
        f'</div>'
        f'<div style="font-style:italic; opacity:0.9; padding:6px 10px; border-radius:6px; border-left:3px solid
            {badge_color}; background:rgba(128,128,128,0.08);">"{span}"</div>'
        f'<div style="font-size:11px; opacity:0.75; margin-top:6px;">{html.escape(grounding.grounding_notes)}</div>'
        f'</div>'
    )
    return badge
```

Notes on this printout: code is reproduced from the project repository without changes to its logic; very long lines are wrapped for print, and emoji characters used in some interface labels are not rendered by the print font. Parts of a file that are not shown are marked with “...” or are outside the selected excerpt.